

Canny Edge Detection

Algorithm Implementation

I smoothed the input image using a two-dimensional Gaussian kernel. I defined this kernel with a window size of 11 pixels and a standard deviation of 2.0. I chose N large enough to capture at least 3 standard deviations on each side, so the kernel does not truncate the Gaussian prematurely. I normalized the kernel elements by dividing them by their sum. I computed the spatial gradients using 3x3 Sobel operators. I convolved the image to filter it horizontally and vertically. I calculated the gradient magnitude using the hypotenuse of the directional gradients.

I calculated the orientation angle using the arctangent function. I built a 256-bin histogram of the gradient magnitudes and calculated the cumulative distribution function to find the thresholds. I set the high threshold at the 85th percentile of the magnitude distribution to represent the non-edge pixels and the low threshold at half the high threshold.

I wrote a quantization method for non-maxima suppression, mapping the gradient angle to 0, 45, 90, or 135 degrees. The method compares the current pixel magnitude to its immediate neighbors along that specific axis. I wrote an interpolation method to evaluate sub-pixel neighbors along the gradient vector. I forced any pixel that was not a local maximum to zero and linked the edges using a depth-first search. I identified all pixels exceeding the high threshold to act as seed points. I searched the 8-connected neighborhood of each seed point. I identified adjacent pixels exceeding the low threshold. I marked these weak edge pixels as valid edges and added them to the search stack.

Results Analysis and Parameter Comparison

- Figure 1 documents the effects of modifying the Gaussian parameters using 6 parameters. I observed that larger kernel sizes and larger standard deviations both increase blurring, though sigma controls the spread of the Gaussian while N only determines how much of it gets sampled. The heavy blur removes background texture in the Lena image. The blur causes adjacent edges to merge.
- I generated Figure 2 to test the percentage of non-edge pixels parameter from 80% to 95%. I found that the higher percentage restricts the output to pixels with the highest contrast. The lower percentage retains more minor details and hence classifies more background noise as valid edges.
- Figure 3 compares the non-maxima suppression methods. The quantization method snaps the gradient angle to one of four discrete axes (0, 45, 90, 135 degrees) and compares against the two grid-aligned neighbors, which introduces dotted artifacts on diagonal edges as visible on the arrow shape in test1. The interpolation method evaluates sub-pixel neighbors along the exact gradient direction using bilinear interpolation, producing smoother and more continuously connected contours on curved and diagonal boundaries. This difference is most visible on the diagonal edges of the arrow shape in test1 and on Lena's hat brim, where quantization leaves periodic gaps while interpolation traces a clean, unbroken line.

- Figure 4 shows the thresholding and edge linking stages. The strong edge map displays disconnected high contrast segments. The weak edge map displays noise and hence magnifies minor details. The final linked output connects the strong segments using the weak edge map.
- Figure 5 compares several edge detectors. I compared the given Canny detector against the Roberts, Zero-crossing, and Sobel detectors. The zero-crossing method detects closed contours but floods the output with background texture because it has no built-in noise suppression beyond the LoG smoothing. Sobel and Roberts both produce thick, filled regions at high-contrast areas like the barcode in test1 because they apply a single global threshold without hysteresis. Canny produces the thinnest edges and best rejects background noise because NMS enforces single-pixel width and hysteresis thresholding discards isolated weak responses that the other detectors retain.